Tuần 1: Làm quen với mô hình master-worker trên ngôn ngữ shell và python của hệ điều hành Ubuntu.

Hiện thực mô hình cluster với 1 master và 2 worker:

Bước 1: Cài đặt môi trường gồm: jdk-14.0.1, apache spark -3.1.1-bin-hadoop2.7 → https://downloads.apache.org

https://sparkbyexamples.com/spark/spark-installation-on-linux-ubuntu/

Bước 1':

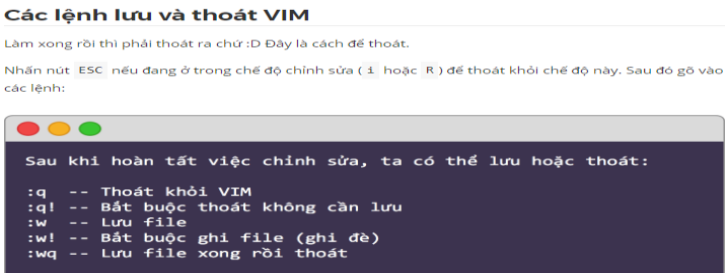
Copy file jdk và apache spark

echo "export JAVA_HOME=/home/hufi/jdk-14.0.1/" >> ~/.bashrc
echo "export PATH=$PATH:$JAVA_HOME/bin" >> ~/.bashrc

Bước 2:

Cài đặt master:
cd spark-3.1.1-bin-hadoop2.7/bin ./spark-class org.apache.spark.deploy.master.Master
Cài đặt worker:
cd spark-3.1.1-bin-hadoop2.7/bin ./spark-class org.apache.spark.deploy.worker.Worker spark://172.17.10.233:7077

Nếu bị lỗi javahome thì sử dụng thêm lệnh:
chmod +x /home/hufi/jdk-14.0.1/bin/*



Bước 3:

Mở terminal mới để Viết chương trình trên scala chuyển cho master thực thi:
cd spark-3.1.1-bin-hadoop2.7/bin ./spark-shell --master spark://172.17.10.233:7077

Bước 4: Kiểm tra kết quả kết nối các worker:

http://localhost:8080 http://localhost:8081

Bài tập:

Bài 1. Đếm từ trong dãy bằng ngôn ngữ shell.

```
val rdd:RDD[Int] = spark.sparkContext.parallelize(List(1,2,3,4,5))
val rddCollect:Array[Int] = rdd.collect()
println("Number of Partitions: "+rdd.getNumPartitions)
println("Action: First element: "+rdd.first())
println("Action: RDD converted to Array[Int] : ")
rddCollect.foreach(println)
rdd.collect().foreach(println)
```

Dựng môi trường jupyter:

Bước 5: Cài đặt biến môi trường cho python:

```
echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.bashrc
echo 'export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.bashrc
echo -e 'if command -v pyenv 1>/dev/null 2>&1; then\n eval "$(pyenv init -)"\nfi' >>
~/.bashrc
echo "export PYSPARK_DRIVER_PYTHON=jupyter" >> ~/.bashrc
echo "export PYSPARK_DRIVER_PYTHON_OPTS='notebook'" >> ~/.bashrc
```

Bước 6: Cài đặt python và jupyter

```
sudo apt-get install -y build-essential libssl-dev zlib1g-dev libbz2-dev \
libreadline-dev libsqlite3-dev wget curl llvm libncurses5-dev libncursesw5-dev \
xz-utils tk-dev libffi-dev liblzma-dev python-openssl git

git clone https://github.com/pyenv/pyenv.git ~/.pyenv
exec "$SHELL" pyenv install 3.9.4
pyenv versions
pyenv global 3.9.4
pip list
pip install pyspark
pip install jupyter
```

Bước 7: Viết chương trình trên python chuyển cho master thực thi:

./pyspark --master spark://172.17.10.252:7077

Bài tập

Bài 2. Tính tổng phần tử trong list và tìm min, max, first, top

Phương pháp 1: aggregate – action

```
spark.sparkContext.setLogLevel("ERROR")
val inputRDD = spark.sparkContext.parallelize(List(("Z", 1),("A", 20),("B", 30),("C", 40),("B", 30),("B", 60)))

val listRdd = spark.sparkContext.parallelize(List(1,2,3,4,5,3,2))

def param0= (accu:Int, v:Int) => accu + v
def param1= (accu1:Int,accu2:Int) => accu1 + accu2
println("aggregate : "+listRdd.aggregate(0)(param0,param1))
//Output: aggregate : 20
//aggregate
def param3= (accu:Int, v:(String,Int)) => accu + v._2
def param4= (accu1:Int,accu2:Int) => accu1 + accu2
println("aggregate : "+inputRDD.aggregate(0)(param3,param4))
```

Phương pháp 2: **treeAggregate – action**

```
def param8= (accu:Int, v:Int) => accu + v
def param9= (accu1:Int,accu2:Int) => accu1 + accu2
println("treeAggregate : "+listRdd.treeAggregate(0)(param8,param9))
```

Phương pháp 3:fold – action

```
//fold
println("fold : "+listRdd.fold(0){ (acc,v) =>
val sum = acc+v
sum
})
//Output: fold : 20

println("fold : "+inputRDD.fold(("Total",0)){(acc:(String,Int),v:(String,Int))=>
val sum = acc._2 + v._2
("Total",sum)
})
//Output: fold : (Total,181)

//min
println("min : "+listRdd.min())
//Output: min : 1
println("min : "+inputRDD.min())
//Output: min : (A,20)

//max
println("max : "+listRdd.max())
//Output: max : 5
println("max : "+inputRDD.max())
//Output: max : (Z,1)
```

Bài 3. Loại các từ trùng nhau trong list

```
val spark = SparkSession.builder()
.appName("SparkByExample")
.master("local")
.getOrCreate()
val rdd = spark.sparkContext.parallelize(
List("Germany India USA","USA India Russia","India Brazil Canada China")
)
val wordsRdd = rdd.flatMap(_.split(" "))
val pairRDD = wordsRdd.map(f=>(f,1))
pairRDD.foreach(println)
```

Bài 4: Đọc file txt, đọc file json, file xml

```
val multiline_df = spark.read.option("multiline","true").json("src/b.json")
multiline_df.show(false)
```